

O Ponto e Vírgula em Go ;

Curso básico de Go

Lexer, tokens e compilação

Errata: vírgulas no bloco `const`

Antes eu disse: precisamos de vírgula após cada constante.

```
const (  
    RegionRN = "rn", // ✗ vírgula desnecessária  
    RegionSP = "sp",  
)
```

Na verdade: nenhuma vírgula ou ponto e vírgula é necessária.

```
const (  
    RegionRN = "rn"  
    RegionSP = "sp"  
)
```

Cada constante em sua linha. O compilador entende sozinho.

Quem faz essa mágica? O **Lexer**!

O **lexer** (ou scanner) é a primeira etapa do compilador.

Ele lê o código caractere por caractere e o transforma em **tokens**.

Código	Tokens gerados
const x=42	[CONST] [IDENT:x] [ASSIGN] [INT:42]

Pense nele como alguém que separa as palavras de uma frase.

Inserção automática de ;

A gramática do Go exige ; no final de cada declaração.

Mas o lexer **insere** ; automaticamente após uma quebra de linha **se** o último token da linha for:

- Um **identificador** (x , int , RegionRN)
- Um **literal** (42 , "rn" , 3.14)
- break , continue , fallthrough , return
- ++ , -- ,) ,] , }

Você quase nunca precisa digitar ; em Go.

Exemplo: o que o lexer faz

Você escreve:

```
const (  
    RegionRN = "rn"  
    RegionSP = "sp"  
)
```

O lexer entrega ao parser:

```
const (  
    RegionRN = "rn";  
    RegionSP = "sp";  
);
```

"rn" é literal → insere ;

) é token da lista → insere ;

Consequência: a chave `{` na mesma linha

```
// ERRO! Lexer insere ; após ")" na linha do if
if x > 0
{
    fmt.Println(x)
}
```

Correto:

```
if x > 0 {
    fmt.Println(x)
}
```

A mesma regra vale para `for`, `func`, `switch`, etc.

Cuidado com expressões multilinha

```
// ERRO! Lexer insere ; após "Hello"  
greeting := "Hello"  
           + "World"
```

Correto: operador no final da linha.

```
greeting := "Hello" +  
            "World"
```

Após `+`, o lexer **não** insere `;`.

Vírgula em literais compostos

Em slices, maps, structs, se o último item está em sua própria linha, a **vírgula final é obrigatória**:

```
regions := []string{  
    "rn",  
    "sp",  
    "mg",  
    "rs", // ← vírgula obrigatória!  
}
```

Sem ela, o lexer insere `;` e o parser quebra.

Uso explícito de ;

Apenas em duas situações comuns:

1. Cláusulas do `for`

```
for i := 0; i < 10; i++ { ... }
```

2. Múltiplas declarações na mesma linha

```
if err := doSomething(); err != nil { ... }
```

Em todo o resto, o lexer cuida. O `gofmt` remove ; extras.

Como o Go compila seu código

Código-fonte



[Lexer] texto → tokens



[Parser] tokens → AST (Abstract Syntax Tree)



[Type Checker] verificação de tipos



[IR] Representação Intermediária



[SSA] Static Single Assignment (otimizações)



[Código de Máquina] instruções da CPU



[Linker] gera executável final

Fases 1–2: Lexer e Parser

- **Lexer**: quebra o código em tokens, insere `;` automaticamente.
- **Parser**: constrói a **AST** – uma árvore que representa a estrutura do programa.

```
File
+-- Package: main
+-- FuncDecl: main
    +-- Body: CallExpr fmt.Println(42)
```

Erros de sintaxe são detectados aqui.

Fases 3–4: Type Checker e IR

- **Type Checker:** valida tipos, resolve variáveis, infere tipos com `:=`.
 - `"hello" + 42` → erro de tipo.
- **IR** (Intermediate Representation): converte a AST para uma forma mais próxima do código de montagem da máquina.

Fases 5–7: SSA, Código de Máquina, Linker

- **SSA** (Static Single Assignment): cada variável é atribuída uma única vez. Permite otimizações fortes (eliminação de código morto, etc.).
- **Código de Máquina**: geração de instruções específicas da arquitetura (amd64, arm64...).
- **Linker**: combina todos os objetos e o runtime em um binário estático.

Tudo isso acontece em frações de segundo – Go compila rápido!

Por que isso importa no nosso código?

Voltando ao exemplo das constantes:

```
const (  
    RegionRN = "rn"    // ← lexer insere ; aqui  
    RegionSP = "sp"    // ← lexer insere ; aqui  
)                    // ← lexer insere ; aqui
```

Graças ao lexer, não precisamos de vírgulas nem `;`.

Ele é nosso amigo!

Dica: visualize o SSA do seu código!

```
GOSSAFUNC=getRegionalVocative go build .
```

Isso gera `ssa.html` – uma página interativa mostrando como o compilador otimiza sua função passo a passo.

Vale a pena explorar!

Recapitulando

- Blocos `const` / `var` **não** precisam de vírgula ou `;` – o lexer insere `;` automaticamente.
- A regra de inserção de `;` exige que `{` fique na mesma linha do `if` / `for`.
- Vírgula final **obrigatória** em literais multilinha.
- Use `;` explicitamente apenas em `for` e múltiplas declarações na mesma linha.
- O compilador Go tem 7 fases, do lexer ao linker, todas otimizadas para velocidade.

Obrigado!

Curso básico de Go

Referências:

go.dev/ref/spec

go.dev/doc/effective_go

go.dev/src/cmd/compile/README